

AI-BASED STUDENT PERFORMANCE PREDICTION AND DATABASE MANAGEMENT SYSTEM

M.Tech Computer Science and Engineering – Mini Project Report

Submitted in partial fulfillment of the requirements for the award of Master of Technology in Computer Science and Engineering.

Academic Year: 2026

PROJECT REPORT

CERTIFICATE

This is to certify that the project entitled **AI-Based Student Performance Prediction and Database Management System** is a bonafide work carried out as part of the M.Tech Computer Science and Engineering programme. The work demonstrates the design of a relational academic database integrated with machine-learning based student performance prediction.

Project Guide: _____

Head of Department: _____

Date: _____

DECLARATION

I hereby declare that this project report entitled **AI-Based Student Performance Prediction and Database Management System** is prepared for academic purposes and represents the work carried out for the project. Any external datasets, algorithms and references used in the study are acknowledged appropriately.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members and institution for their guidance and support. I also acknowledge the open-source Python, Flask, MySQL and Scikit-learn communities whose tools support the implementation of this project.

ABSTRACT

Educational institutions maintain large quantities of student information such as attendance, internal assessment, assignment marks, examination performance and previous academic records. Conventional database systems provide storage and retrieval but generally do not provide predictive insight into future performance. This project proposes an integrated database management and artificial intelligence system for student performance prediction.

The system uses MySQL for structured storage and a Python Flask application for interaction. Machine-learning models including Logistic Regression, Decision Tree and Random Forest are trained using academic features such as attendance percentage, internal marks, assignment marks, previous GPA and previous failures. The output is classified into High, Average or Low performance. Prediction history is stored in the database for later analysis.

The proposed system can support early identification of students who may require academic intervention. The supplied implementation is a demonstration framework; final institutional evaluation should use an approved real dataset and report the actual experimental measurements.

Keywords

Artificial Intelligence, Machine Learning, Student Performance Prediction, MySQL, Flask, Random Forest, Educational Data Mining, Database Management System.

TABLE OF CONTENTS

1. Introduction	
2. Literature Review	
3. Existing System and Research Gap	
4. Proposed System	
5. System Requirements	
6. Database Design	
7. System Architecture and UML	
8. Machine Learning Methodology	
9. Implementation	
10. Experimental Evaluation	
11. Advantages, Limitations and Applications	
12. Future Scope	
13. Conclusion	
References	
Appendix A – Core Code	
Appendix B – Viva Questions	

CHAPTER 1 – INTRODUCTION

1.1 Background

Modern educational institutions generate academic data continuously. Attendance records, assessment marks, assignment scores, semester GPA and course outcomes can be used not only for reporting but also for predictive analytics. Educational data mining applies computational techniques to discover useful patterns in this information.

1.2 Problem Statement

A conventional student database can answer questions about past records but may not automatically identify students at risk of poor performance. The problem addressed here is to design a database-driven application that stores academic information and uses machine learning to predict student performance categories.

1.3 Objectives

- Design a structured relational database for academic records.
- Develop a web-based interface using Flask.
- Preprocess and prepare student performance data.
- Train and compare multiple classification algorithms.
- Predict High, Average or Low performance.
- Store prediction results for monitoring and analysis.

1.4 Scope

The system is intended as an academic prototype. It can be extended to university ERP systems, learning-management systems and institutional analytics platforms.

CHAPTER 2 – LITERATURE REVIEW

2.1 Educational Data Mining

Educational data mining uses statistical and machine-learning methods to analyze learner information. Common tasks include grade prediction, dropout prediction, clustering and recommendation.

2.2 Machine Learning for Performance Prediction

Classification algorithms such as Logistic Regression, Decision Trees, Random Forests and support-vector methods can learn relationships between academic indicators and performance classes. Ensemble methods can capture nonlinear interactions among features.

2.3 Database Integration

A useful academic information system requires persistent storage, data integrity, controlled relationships and query capability. Integrating an ML layer with a relational database allows predictions to be associated with individual students and retained for audit and longitudinal analysis.

2.4 Research Gap

Many simple student-management applications stop at data storage and reporting. A more useful system combines normalized academic records, predictive modelling, prediction history and a user-facing dashboard. Another important gap is generalization: model performance can change when institutional data differs from the training data. Therefore, real deployment requires validation on representative data.

CHAPTER 3 – EXISTING SYSTEM AND RESEARCH GAP

3.1 Existing System

- Manual calculation of performance indicators.
- Static database reports.
- Limited early-warning capability.
- Separate tools for data management and analysis.
- Difficult to monitor prediction history.

3.2 Limitations

Manual analysis is time-consuming and may delay intervention. Static reports summarize past performance but do not directly estimate likely future outcomes.

3.3 Proposed Improvement

The proposed system joins academic data management and machine learning in one application. Academic features are passed to a trained model and the resulting classification is stored alongside the student record.

CHAPTER 4 – PROPOSED SYSTEM

4.1 Overview

The system consists of a web interface, application layer, relational database and ML prediction component. An administrator can manage student records and submit academic indicators to obtain a prediction.

4.2 Main Modules

- Authentication module
- Student management module
- Subject management module
- Attendance and marks management
- Performance management
- ML training and prediction
- Prediction history
- Dashboard and reporting

4.3 Prediction Classes

High: expected strong academic performance. **Average:** expected moderate performance. **Low:** potential academic risk requiring attention. These labels are model-defined categories and should be calibrated using the chosen institutional dataset.

CHAPTER 5 – SYSTEM REQUIREMENTS

5.1 Hardware

- Processor: modern dual-core or better
- RAM: 4 GB minimum; 8 GB recommended
- Storage: 2 GB or more for project and Python environment
- Network: optional for local deployment

5.2 Software

- Windows/Linux/macOS
- Python 3.10 or later
- MySQL 8.x or compatible version
- Flask
- Pandas and NumPy
- Scikit-learn
- Joblib
- VS Code or another Python IDE

5.3 Functional Requirements

- Authenticate administrator.
- Create and view student records.
- Store academic information.
- Train ML models.
- Predict performance.
- Store prediction history.
- Display summary statistics.

5.4 Non-functional Requirements

- Usability
- Reliability
- Security
- Maintainability
- Scalability
- Data integrity

CHAPTER 6 – DATABASE DESIGN

6.1 Entities

The main entities are Students, Subjects, Attendance, Marks, Performance and Predictions.

Table	Primary Key	Purpose
students	student_id	Student master data
subjects	subject_id	Course information
attendance	attendance_id	Attendance records
marks	mark_id	Assessment marks
performance	performance_id	GPA/grade
predictions	prediction_id	AI prediction history

6.2 Normalization

The schema separates student, subject and transactional academic information to reduce redundancy. Foreign keys maintain relationships between records. Further normalization can be applied according to institutional requirements.

6.3 SQL Operations

The implementation includes CREATE DATABASE, CREATE TABLE, INSERT and foreign-key definitions. Indexes can be added to frequently queried foreign-key columns in larger deployments.

CHAPTER 7 – SYSTEM ARCHITECTURE AND UML

7.1 Architecture

Student/User → Flask Web Interface → Application Layer → MySQL Database

Application Layer → Preprocessing → ML Model → Prediction → MySQL Prediction History

7.2 Use Cases

- Administrator logs in.
- Administrator adds or views students.
- Administrator enters academic indicators.
- System preprocesses input.
- System predicts performance.
- System stores prediction.
- Administrator reviews dashboard.

7.3 Sequence

The user submits an input form. Flask validates the request, loads the serialized ML model, constructs the feature vector, performs prediction and stores the result in MySQL. The predicted class and confidence estimate are returned to the interface.

7.4 Class Components

Core software components are Student, Preprocessor, FeatureExtractor, Classifier, Prediction and DatabaseManager. The current prototype keeps these responsibilities compact; a production implementation should separate them into service classes.

CHAPTER 8 – MACHINE LEARNING METHODOLOGY

8.1 Input Features

- Attendance percentage
- Internal assessment marks
- Assignment marks
- Previous GPA
- Number of previous failures

8.2 Preprocessing

The dataset is checked for missing values, valid numeric ranges and consistent class labels. Features are separated from the target variable. A stratified 80/20 train-test split is used in the demonstration.

8.3 Logistic Regression

Logistic Regression estimates class probabilities from a weighted combination of input features. Standardization is used in the supplied implementation.

8.4 Decision Tree

A Decision Tree recursively divides the feature space into regions that improve class separation. It is easy to interpret but may overfit if tree complexity is not controlled.

8.5 Random Forest

Random Forest combines many decision trees trained with randomized samples/features. It is selected as the deployable demonstration model because ensemble averaging can improve robustness compared with a single tree.

8.6 Evaluation Metrics

Accuracy measures overall correctness. Precision measures correctness among predicted instances of a class. Recall measures coverage of actual instances. F1-score combines precision and recall through their harmonic mean.

CHAPTER 9 – IMPLEMENTATION

9.1 Technology Stack

Python is used for application and ML development. Flask provides the web layer, MySQL provides relational storage, Pandas handles tabular data and Scikit-learn provides the classification algorithms.

9.2 Model Training Workflow

The training script reads the CSV dataset, selects five features, performs a stratified split, trains three classifiers, calculates evaluation metrics and saves the Random Forest model using Joblib.

9.3 Prediction Workflow

The prediction page collects attendance, internal marks, assignment marks, previous GPA and previous failures. The serialized model produces a class and probability. The result is inserted into the predictions table.

9.4 Installation

- Install Python and MySQL.
- Create the database by executing database/student_performance.sql.
- Install packages using `pip install -r requirements.txt`.
- Update MySQL credentials in `app.py`.
- Run `python ml/train_model.py`.
- Run `python app.py`.
- Open `http://127.0.0.1:5000/`.

9.5 Demonstration Login

Username: admin | Password: admin123. Change these credentials and secret key before any real deployment.

CHAPTER 10 – EXPERIMENTAL EVALUATION

10.1 Dataset

The supplied project package contains a small synthetic demonstration CSV. It is included only to make the software pipeline reproducible. It must not be described as an institutional dataset.

10.2 Experimental Procedure

The data is divided using an 80/20 stratified split. Three models are trained using the same training set. Weighted precision, recall and F1-score are calculated on the test set. The Random Forest model is serialized for application use.

10.3 Results Table

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	Run script	Run script	Run script	Run script
Decision Tree	Run script	Run script	Run script	Run script
Random Forest	Run script	Run script	Run script	Run script

Run `python ml/train_model.py` to generate reports/model_comparison.csv. Insert those actual values into the final submitted report.

10.4 Confusion Matrix

A confusion matrix should be generated from the selected test set to show correct and incorrect predictions for High, Average and Low classes. This helps identify which class is hardest to predict.

10.5 Interpretation

Model comparison should consider more than accuracy, particularly if class frequencies are unequal. For an early-warning system, recall for the Low-performance class may be especially important because missing an at-risk student can be more consequential than generating an additional review case.

CHAPTER 11 – ADVANTAGES, LIMITATIONS AND APPLICATIONS

11.1 Advantages

- Combines DBMS and AI in one application.
- Automates performance classification.
- Supports early identification of potentially at-risk students.
- Stores prediction history.
- Can be extended to institutional systems.

11.2 Limitations

- Predictions depend on training-data quality.
- Synthetic demonstration data is not suitable for institutional conclusions.
- Academic performance is influenced by factors not included in the model.
- Production use requires stronger authentication, authorization and privacy controls.
- Model drift may occur when student populations or curricula change.

11.3 Applications

- Academic advising
- Early-warning systems
- Department dashboards
- Student support programmes
- Institutional analytics

CHAPTER 12 – FUTURE SCOPE

- XGBoost and gradient-boosting models.
- BERT/transformer-based educational text analysis where appropriate.
- Explainable AI using feature importance or SHAP.
- Real-time LMS/ERP integration.
- Automated alerts for academic advisors.
- Time-series semester performance prediction.
- Fairness and subgroup performance monitoring.
- Cloud deployment with secure APIs.
- Role-based access control and audit logging.

CHAPTER 13 – CONCLUSION

This project demonstrates an integrated approach to student information management and academic performance prediction. A MySQL relational database provides persistent structured storage while Flask connects users to the application and machine-learning components. Logistic Regression, Decision Tree and Random Forest provide a practical model-comparison framework.

The system can support academic decision-making by identifying performance categories from selected indicators. For a final institutional project, the next essential step is evaluation with an approved representative dataset, rigorous cross-validation or external validation, security hardening and transparent documentation of the actual results.

REFERENCES

1. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
2. W. McKinney, Python for Data Analysis, O'Reilly Media.
3. M. Grinberg, Flask Web Development, O'Reilly Media.
4. Oracle, MySQL 8.0 Reference Manual.
5. T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, Springer.
6. L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001.
7. C. M. Bishop, Pattern Recognition and Machine Learning, Springer.
8. P. Cortez and A. Silva, "Using Data Mining to Predict Secondary School Student Performance," in Proc. 5th FUTURE Business Technology Conference, 2008.

APPENDIX A – CORE PYTHON CODE

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv("ml/student_performance.csv")
X = df[["attendance", "internal", "assignment", "gpa", "failures"]]
y = df["performance"]

Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

models = {
    "Logistic Regression": Pipeline([
        ("scale", StandardScaler()),
        ("clf", LogisticRegression(max_iter=2000))
    ]),
    "Decision Tree": DecisionTreeClassifier(max_depth=6, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=300, random_state=42)
}
```

APPENDIX B – VIVA QUESTIONS AND ANSWERS

What is the main objective?

To integrate student academic database management with ML-based performance prediction.

Why use Random Forest?

It combines multiple decision trees and can model nonlinear feature relationships.

What is a feature?

An input variable used by the ML model.

What is classification?

Assigning an observation to one of predefined categories.

What are the output classes?

High, Average and Low.

Why MySQL?

It provides reliable relational storage, SQL querying and integrity constraints.

Why Flask?

It provides a lightweight Python web framework for integrating the UI and ML code.

What is accuracy?

The proportion of correct predictions among all predictions.

What is precision?

The proportion of predicted positives that are actually positive.

What is recall?

The proportion of actual positives correctly identified.

What is F1-score?

The harmonic mean of precision and recall.

What is overfitting?

When a model learns training-specific patterns and generalizes poorly.

What is a foreign key?

A column that references a key in another table.

Why store predictions?

For history, auditing and future analysis.

Can the model guarantee performance?

No. It provides a probabilistic estimate dependent on data.

What is future scope?

Explainable AI, stronger models, ERP integration and real-time monitoring.